

P1 – DevOps e Infraestrutura Privada

Checkpoint 01 · Projeto Burnout (Grupo 6 – UNIFEOP)

Fluxo	OpenTofu cria a VM → cloud-init configura usuário e SSH → Ansible prepara o ambiente → simulador é enviado por SCP → roda na VM e grava o CSV
Problema	Monitoramento de clima organizacional e risco de burnout por colaborador
Repositório	github.com/calebe-moraes/Projeto_Burnout_DataScience (branch main)

5.1 Simulador mínimo de dados (1,00 ponto)

Um script Python que gera dados fictícios de exaustão de colaboradores.

- Código em **simulador/simulador.py**; dependências em **simulador/requirements.txt**.
- Roda por um comando, sem interface gráfica: `python simulador.py --quantidade 20`.
- Cada registro tem: **id_registro**, **id_colaborador**, **departamento**, **horas_extras**, **horas_trabalhadas**, **tickets_fora_horario**, **nivel_clima**, **risco_burnout** e **data_hora_geracao** (mais de 3 variáveis + data/hora).
- Gera no mínimo 10 registros por execução (padrão 20) e salva em CSV.
- Exemplo de saída em **dados/exemplo_dados.csv**.

Como comprovar:

```
python simulador/simulador.py --quantidade 15 --saida dados/exemplo_dados.csv
```

5.2 Provisionamento da VM com OpenTofu (1,25 ponto)

A VM Ubuntu 22.04 é criada por código, no libvirt/KVM da máquina hospedeira.

- Arquivos **infraestrutura/main.tf** e **variables.tf** (provider libvirt): imagem base, disco, disco do cloud-init, VM e rede.
- O cloud-init é ligado à VM pelo recurso `libvirt_cloudinit_disk`.
- O OpenTofu também gera o **ansible/inventory.ini** com o IP real da VM.
- Sem KVM, a variável `vm_virtualizacao = "qemu"` usa emulação.

Como comprovar:

```
cd infraestrutura
tofu init
tofu apply
tofu output # mostra o IP da VM
virsh -c qemu:///system list --all
```

5.3 Configuração inicial com cloud-init (0,75 ponto)

Aplicado automaticamente no primeiro boot da VM.

- Arquivo **infraestrutura/cloud_init.cfg**, renderizado pelo OpenTofu.
- Cria o usuário administrador (sudo sem senha) e instala a **chave pública SSH**.
- Desliga login por senha e login de root: só entra com a chave.
- Grava `/etc/burnout-cloud-init.txt` como prova de que foi aplicado.

Como comprovar:

```
./scripts/verificar.sh # mostra usuário, sudo, marcador, cloud-init status e PasswordAuthentication
```

5.4 Configuração do ambiente com Ansible (1,25 ponto)

Prepara a VM para rodar o simulador. Pode ser executado várias vezes sem erro.

- **infraestrutura/ansible/**: inventory.ini, playbook.yml e ansible.cfg.
- O inventário tem IP, usuário e chave SSH da VM.
- O playbook instala python3, python3-venv e python3-pip, cria o venv e instala as bibliotecas do requirements.txt.
- Cria */opt/burnout/simulador* e */opt/burnout/dados* com o dono correto.
- É idempotente: na segunda execução aparece **changed=0**.

Como comprovar:

```
cd infraestrutura/ansible
ansible-playbook -i inventory.ini playbook.yml
ansible-playbook -i inventory.ini playbook.yml # 2a vez: changed=0
```

5.5 Implantação da aplicação por SSH (0,75 ponto)

O código do simulador sai da hospedeira e vai para a VM por SCP.

- Script **infraestrutura/scripts/deploy.sh**: copia *simulador/* para */opt/burnout/simulador* usando SCP com a chave SSH.
- Depois lista os arquivos no destino para comprovar a cópia.
- O simulador não usa caminhos da hospedeira: o destino dos dados vem da variável BURNOUT_DADOS_DIR.

Como comprovar:

```
./scripts/deploy.sh
ssh -i ~/.ssh/id_ed25519 burnout@$(tofu output -raw vm_ip) 'ls -l
/opt/burnout/simulador'
```

5.6 Execução e geração de dados na VM (1,25 ponto)

O simulador roda dentro da VM e grava o CSV nela mesma.

- Script **infraestrutura/scripts/executar.sh**: roda o simulador no venv criado pelo Ansible.
- Os dados ficam em */opt/burnout/dados/dados_burnout.csv*, dentro da VM.
- Cada execução **acrescenta** novos registros (modo append) e não apaga os anteriores.
- Os campos (horas extras, tickets fora do horário, clima, risco) correspondem ao problema de burnout.

Como comprovar:

```
./scripts/executar.sh 20
./scripts/executar.sh 20 # o total de linhas do CSV deve crescer
```

5.7 Repositório GitHub e documentação (0,75 ponto)

Tudo versionado e separado por pasta.

- **infraestrutura/** (OpenTofu, cloud-init, Ansible, scripts), **simulador/** (aplicação) e **dados/** (exemplo) estão em pastas separadas.
- O **README.md** tem requisitos, configuração local e o passo a passo de provisionar, configurar, enviar e executar.
- O histórico tem commits separados por assunto (simulador, OpenTofu/cloud-init, Ansible/scripts, segurança, documentação).
- Nada sensível no repositório: *.gitignore* bloqueia tfstate, tfvars e chaves; a senha do Docker saiu do código e vai para o *.env* (modelo em *.env.example*); só a chave pública vai à VM.

Como comprovar:

```
git log --oneline
git ls-files | grep -E 'tfstate|tfvars$|id_ed25519|\.env$' # não deve listar nada
```

Observação: o código está pronto e versionado. A execução na VM (itens 5.2 a 5.6) é comprovada na demonstração com os comandos acima.